

# Guide d'apprentissage — OS-pour-IA

---

**Pour qui ?** Toute personne qui découvre ce projet et souhaite *tout* comprendre, sans rien tenir pour acquis. On part de zéro. Chaque mot technique est défini la première fois qu'il apparaît, et chaque décision est justifiée — pas seulement énoncée.

**Comment lire ?** Dans l'ordre. Chaque section s'appuie sur la précédente. Les encadrés > 💡 donnent une analogie ou une reformulation simple. Les encadrés > ⚠️ signalent un piège ou une nuance souvent confondue.

**Une note de vocabulaire.** Le projet est écrit en anglais dans son code et sa spécification. Ce guide privilégie le terme français ; lorsqu'un terme technique anglais est incontournable (parce qu'il figure tel quel dans le code), il est rappelé entre parenthèses, afin de pouvoir relier ce guide aux fichiers sources.

💡 **En une phrase :** ce projet se demande à quoi ressemblerait un système d'exploitation si ses utilisateurs principaux n'étaient pas des humains, mais des programmes d'IA autonomes — puis il en construit et teste une partie réelle.

---

## Table des matières

---

1. Les mots de base : système d'exploitation, noyau, agent
  2. Le problème : pourquoi Linux ne convient pas aux agents IA
  3. Qui est l'utilisateur cible : le « profil B »
  4. Le vocabulaire du projet, défini une fois pour toutes
  5. La thèse : trois (puis six) propriétés mesurables
  6. L'architecture : les pièces et leur agencement
  7. Le parcours d'une action, de bout en bout
  8. Les cinq paris architecturaux, et leur justification
  9. Les deux substrats : Linux puis seL4
  10. La méthode : comment on établit que c'est vrai
  11. Ce qui est démontré, ce qui ne l'est pas
  12. Glossaire de poche
-

# 1. Les mots de base

---

Avant de parler du projet, trois mots sont nécessaires.

## Un système d'exploitation

Un **système d'exploitation** (en anglais *operating system*, abrégé OS) est le programme qui se tient entre le matériel (processeur, mémoire, disque, carte réseau) et les programmes que l'on exécute. Lorsqu'un traitement de texte veut enregistrer un fichier, il ne s'adresse pas directement au disque dur : il demande au système d'exploitation de le faire. Celui-ci arbitre, protège et partage les ressources entre tous les programmes.

Exemples : **Linux**, **Windows**, **macOS**.

💡 On peut se représenter le système d'exploitation comme le gérant d'un immeuble. Les locataires (les programmes) ne touchent jamais directement à la plomberie ni au compteur électrique. Ils passent par le gérant, qui décide qui a accès à quoi.

## Le noyau

Le **noyau** (en anglais *kernel*) est le cœur du système d'exploitation : la partie qui a tous les pouvoirs sur le matériel. C'est lui qui décide quel programme s'exécute à quel instant, quelle zone mémoire est lisible par qui, etc. Quand on dit « Linux », on désigne techniquement surtout son noyau.

⚠️ **Point essentiel pour ce projet** : ce projet **n'écrit pas de noyau**. Il construit un environnement d'exécution (voir ci-dessous) qui s'installe **par-dessus** un noyau existant (Linux, ou seL4). Ce n'est pas un système d'exploitation au sens « il démarre la machine » — c'est un système d'exploitation au sens « il offre aux programmes les services qu'un système d'exploitation offre ».

## Un environnement d'exécution

Un **environnement d'exécution** (en anglais *runtime*) est un programme qui héberge et fait fonctionner d'autres programmes, en leur fournissant un cadre et des services. L'environnement d'exécution de ce projet héberge des agents IA et leur offre des services « de classe système d'exploitation » (mémoire, traçabilité, contrôle d'accès), sans être lui-même le noyau de la machine.

💡 Du point de vue d'un agent, cet environnement d'exécution est son système d'exploitation : l'agent ne voit jamais Linux directement, seulement les abstractions que l'environnement lui présente.

## Un agent IA

Dans ce projet, un **agent** est un programme piloté par une IA — typiquement un grand modèle de langage (en anglais *Large Language Model*, abrégé LLM, comme ceux qui animent ChatGPT ou Claude) — qui exécute des tâches de façon **autonome** : il reçoit un objectif, raisonne, agit sur le système, observe le résultat, recommence — sans qu'un humain valide chaque étape.

Des exemples concrets qui existent aujourd'hui : Claude Code, Devin, SWE-agent. Ils fonctionnent pendant des heures, exécutent des milliers d'opérations, avec une supervision humaine minimale.

---

## 2. Le problème

---

**Les systèmes d'exploitation actuels ont été conçus entre 1960 et 1990, pour des humains assis devant un terminal.** Ces hypothèses n'ont jamais été écrites noir sur blanc — elles semblaient évidentes. Elles ne le sont plus dès que l'utilisateur est un agent IA.

Voici les sept hypothèses implicites, et ce qu'elles coûtent à un agent.

### H-1 — « L'utilisateur perçoit le temps en millisecondes »

Linux est réglé pour qu'un humain *ressente* le système comme réactif : il découpe le temps du processeur en tranches de quelques millisecondes pour donner l'illusion que tout fonctionne simultanément.

**Coût pour un agent** : faire fonctionner 1 000 agents à la fois, chacun comme un programme classique, oblige le système à passer une part énorme de son temps à jongler entre eux. À 1 000 agents, ce surcoût de jonglage peut consommer un cœur de processeur entier — pour ne rien produire d'utile.

### H-2 — « L'interface est du texte lu par un humain »

Sous Unix, « tout est un fichier » et tout se lit en texte. La commande `ls` liste des fichiers en texte ; `ps` liste les programmes en texte. Ce texte est destiné à un œil humain, pas à une analyse par une machine.

**Coût pour un agent** : l'agent doit *analyser* ce texte (en anglais *parsing*). Or le format change selon la version du système, la langue configurée, l'encodage... C'est fragile, et c'est une source entière de bugs évitables.

### H-3 — « L'état du système s'accumule implicitement »

Il n'existe aucune commande disant « donner une photographie complète et cohérente de l'état du système maintenant ». L'état est éparpillé : fichiers modifiés, mémoire des programmes, connexions réseau ouvertes... Un humain reconstruit mentalement ce qui s'est passé.

**Coût pour un agent** : il est impossible de prendre une « photographie » (un *instantané*, en anglais *snapshot*) propre de son état pour pouvoir y revenir plus tard. Donc impossible d'**annuler** proprement une erreur. (C'est l'objet de la propriété P2.)

### H-4 — « La confiance est liée à l'identité »

Sous Unix, les droits dépendent de *qui l'on est* (l'identité utilisateur). L'identité « root » (l'administrateur) confère tous les droits, tout le temps, sur tout.

**Coût pour un agent** : un agent IA n'a pas une identité stable et prévisible — son comportement dépend de ce qu'on lui demande et de la part d'aléa du modèle. Lui donner « tous les droits parce qu'on lui fait confiance » revient à confier les clés de la maison à une entité dont on ne sait pas ce qu'elle fera la minute suivante. Et il n'existe aucun moyen *natif* de dire « voici le droit de lire ce fichier précis, seulement pour cinq minutes, et révoquant à tout instant ».

### H-5 — « Le parallélisme est borné par ce qu'un humain peut superviser »

Les outils système affichent quelques dizaines à quelques centaines de programmes — ce qu'un humain peut surveiller. Chaque programme porte un coût fixe : de la mémoire réservée rien que pour exister.

**Coût pour un agent** : sur une machine de 16 Go, un conteneur Docker par agent (la pratique courante, voir ci-dessous) coûte de 100 Mo à 1 Go *chacun*. Résultat : quelques dizaines d'agents, pas des milliers. Le coût n'est pas physique — il provient des abstractions conçues pour un petit nombre.

 **Docker / conteneur** : une technologie qui emballe un programme avec tout ce dont il a besoin (bibliothèques, fichiers) dans une « boîte » isolée des autres. Pratique, mais lourde : chaque boîte embarque une copie de beaucoup de choses.

### H-6 — « La persistance est modifiable et sans historique »

Lorsqu'on écrase un fichier, l'ancienne version est perdue. Le système de fichiers ne conserve pas d'historique par défaut.

**Coût pour un agent** : chaque agent doit réinventer sa propre logique de sauvegarde/restauration. C'est du travail dupliqué, incohérent d'un agent à l'autre, et sans garantie : une panne au mauvais moment peut laisser un état à moitié écrit.

## H-7 — « L'observabilité est reconstructive »

Pour comprendre *pourquoi* un événement s'est produit sous Linux, il faut croiser plusieurs journaux (en anglais *logs*) séparés, à la main, après coup. C'est lent et incomplet.

**Coût pour un agent** : pour un agent ayant exécuté dix millions d'opérations, retrouver la chaîne de causes d'une opération précise n'est pas une opération rapide réalisable à la demande. Or auditer un agent autonome, c'est exactement cela.

### Le signal qui confirme le problème

La preuve qu'il manque quelque chose au niveau du système d'exploitation ? **Les développeurs le réinventent au niveau applicatif**. Des outils comme **Temporal** ou **LangGraph** réimplémentent, au prix fort, dans chaque application, des fonctions qui *devraient* être des services système : l'annulation, le rejeu déterministe, la gestion d'état explicite.

💡 C'est un schéma classique en informatique : une fonctionnalité est d'abord bricolée dans chaque application, puis finit par descendre dans le système quand on comprend que c'est sa place. Le ramasse-miettes mémoire, les bases de données embarquées, les conteneurs : tous ont suivi ce chemin.

**La question du projet** : quelles abstractions de niveau système d'exploitation feraient disparaître cette couche de bricolage ?

---

## 3. Le profil B

On ne peut pas concevoir pour « les agents IA » en général — ils sont trop différents. Le projet se donne donc une cible précise, appelée **profil B** :

| Critère          | Valeur                                                                                    |
|------------------|-------------------------------------------------------------------------------------------|
| (a) Durée de vie | de 1 heure à 1 mois — ni éphémère, ni éternel                                             |
| (b) État         | persistant entre les actions (il ne repart pas de zéro à chaque fois)                     |
| (c) Volume       | de $10^4$ à $10^8$ actions sur sa vie entière                                             |
| (d) Supervision  | un humain intervient <i>ponctuellement</i> (toutes les heures/jours), pas à chaque action |

| Critère | Valeur |
|---------|--------|
|---------|--------|

**(e) Délégation** l'agent peut créer des **sous-agents** dotés d'une partie de ses droits

💡 Le profil B se distingue de trois autres usages : ce n'est ni un agent par lots (en anglais *batch*, qui vit quelques secondes et oublie tout), ni un *service permanent* (qui fonctionne indéfiniment sans point de contrôle), ni un agent *interactif* (où un humain valide chaque pas).

## Une honnêteté importante : l'instrument de mesure fait partie de la cage

Les grands modèles de langage raisonnent *comme* des humains : lentement, en langage, à un rythme de quelques actions par seconde au plus. Le projet en est conscient et le dit clairement : en calibrant tout sur ces modèles, on risque de concevoir un système d'exploitation « pour des humains qui ne dorment jamais » plutôt que pour des agents réellement différents — un agent d'apprentissage par renforcement émet *un million* d'actions par seconde, et les seuils du projet n'ont alors aucun sens.

Le projet sépare donc : - **Ce qui est invariant** (vrai pour tout agent autonome supervisé) : il faut un historique des causes, un moyen d'annuler, et une délégation révocable des droits. - **Ce qui est dimensionné pour les modèles d'aujourd'hui** : les seuils chiffrés (10 ms, 100 ms, le corridor 1 h – 1 mois). À réviser quand d'autres types d'agents domineront.

💡 Pourquoi tester sur des grands modèles de langage, alors ? Parce qu'ils sont **lents et bavards** : ils verbalisent leur raisonnement, donc lorsqu'un défaut de conception existe, ils échouent de façon *visible et compréhensible*. Ce sont de mauvais sujets pour mesurer la performance, mais d'excellents sujets pour révéler les défauts de conception.

## 4. Vocabulaire

Ces termes reviennent partout. Il vaut la peine de les assimiler : la suite du document les emploie sans les redéfinir.

### Action

L'**unité de base** du projet. Une action = un message reçu et traité par un agent, **ou** un message émis par un agent, telle que l'environnement d'exécution l'observe.

⚠️ Une action n'est *pas* une instruction machine, ni un appel de fonction, ni une requête réseau. C'est un **échange de message entre acteurs**. Un calcul purement interne, sans message, n'est pas une action ici — c'est un choix délibéré : on observe les échanges, pas chaque rouage interne.

## Acteur / Agent

Un **acteur** est une entité qui reçoit des messages, les traite un par un, puis émet des messages. Un **agent** est un acteur identifié par un identifiant permanent (`agent_id`).

Point clé : **un agent est mono-tâche en interne**. Il n'a pas de fils d'exécution (en anglais *threads*) parallèles. Pour faire plusieurs choses à la fois, il **engendre** (en anglais *spawn*) des sous-agents. La concurrence se fait *entre* agents, jamais à *l'intérieur* d'un agent.

💡 Pourquoi ce choix ? Parce que la concurrence interne (plusieurs fils d'exécution qui partagent la même mémoire) est la principale source de bugs impossibles à reproduire. En l'interdisant, on rend chaque agent déterministe et rejouable (voir P5).

Autre point clé : une panne suivie d'un redémarrage **ne crée pas un nouvel agent**. C'est le même `agent_id` qui reprend depuis son dernier état sauvegardé. L'identité d'un agent n'est pas son numéro de processus (en anglais *PID*, *Process Identifier*) — c'est son `agent_id`, qui survit aux redémarrages.

## Capacité (capability)

Une **capacité** (en anglais *capability* — le terme du code et de la littérature) est un **jeton conférant un droit précis**. C'est le cœur du modèle de sécurité du projet. Elle possède quatre propriétés :

- **Non-ambiante** : aucun droit par défaut. Pour effectuer une opération, l'acteur doit *détenir* le jeton qui l'y autorise. Pas de jeton = pas d'accès, point.
- **Délégable** : un acteur peut transmettre un jeton (ou une version réduite) à un autre acteur.
- **Dérivable / atténuable** : on peut créer une version *plus restreinte* d'un jeton (moins de droits, portée plus étroite). **Jamais l'inverse**. Un sous-agent ne peut pas disposer de plus de droits que son parent.
- **Révocable** : on peut retirer un jeton à tout moment ; cela invalide **en cascade** tous les jetons qui en dérivent.

💡 À comparer au modèle Unix « root a tous les droits ». Avec les capacités, on dit : « voici le droit de lire ce fichier précis, retirable à volonté ». C'est l'inverse du « tout ou rien ». Cette idée n'est pas neuve (elle date de 1966) ; le projet l'applique à l'environnement d'exécution d'agents IA, avec la nouveauté de la délégation/révocation **dynamique** entre sous-agents créés en cours d'exécution.

## Barrière de validation (commit barrier)

Une **barrière de validation** (en anglais *commit barrier*) est un **point de non-retour**. Une fois franchie, les actions qui la précèdent ne peuvent plus être annulées.

Pourquoi en a-t-on besoin ? Parce que certaines actions sont *réellement* irréversibles : une fois qu'un

paquet réseau a quitté la machine, on ne peut pas le rappeler. La barrière marque la frontière entre « ce qu'on peut encore défaire » et « ce qui est gravé dans le marbre ».

Le mécanisme est **hybride conservateur** : - **Automatique** pour une courte liste d'effets prouvablement irréversibles (un paquet réseau effectivement parti, une écriture sur un disque externe). - **Explicite** (appel `commit()`) pour tout le reste — c'est le cas normal. - **Garde-fou** : si un agent tente un effet externe sans avoir posé de barrière, le système refuse et suspend l'action jusqu'à ce que l'agent tranche (valider ou annuler).

## Transaction

Une **transaction** est simplement la séquence d'actions **comprises entre deux barrières de validation**. C'est l'unité de l'annulation : soit toutes les actions d'une transaction sont annulées, soit aucune (jamais d'état à moitié défait).

## Retour arrière (rollback)

Le **retour arrière** (en anglais *rollback*) est l'opération qui **restaure l'état local** tel qu'il était à un instant passé (mais après la dernière barrière de validation seulement).

⚠ Le retour arrière **n'est pas** une « compensation ». Il ne rappelle pas les services externes, n'envoie pas de messages d'annulation. Il restaure l'état *local* de la machine. Annuler des effets partis vers l'extérieur est un problème non résolu en général ; le projet l'exclut explicitement.

## État local

Ce que le retour arrière peut restaurer : l'**état local** = l'état des acteurs locaux + le magasin de données local + les messages encore en transit à l'*intérieur* du nœud. Cela **exclut** tout ce qui est parti vers l'extérieur (réseau, services tiers, autres machines).

## Adressé par le contenu (content-addressed)

Concept central, et un peu déroutant au début. Habituellement, on range une donnée à une *adresse* qu'on choisit (« case n°5 »). En mode **adressé par le contenu** (en anglais *content-addressed*), l'adresse d'une donnée **est calculée à partir de son contenu**, via une fonction de hachage.

💡 **Hachage / empreinte (hash)** : une fonction qui transforme n'importe quelle donnée en une empreinte de taille fixe (ici 32 octets, avec l'algorithme SHA-256). La même donnée donne toujours la même empreinte ; deux données différentes donnent (en pratique) des empreintes différentes. C'est comparable à une empreinte digitale du contenu.

Conséquences remarquables : - L'identifiant est **déterministe** (le contenu détermine l'adresse). - Il est



**infalsifiable** (impossible de fabriquer une donnée qui colle à une empreinte choisie d'avance). - Les versions successives ne s'écrasent pas : chaque version a sa propre empreinte, donc **l'historique est conservé gratuitement**. C'est précisément ce qui rend le retour arrière possible. (C'est aussi ainsi que fonctionne Git.)

## Superviseur asymétrique

Le **rôle humain** dans le système. « Asymétrique » parce qu'il **n'est pas dans la boucle à chaque action**. Il dispose de trois pouvoirs, via des capacités privilégiées : - **Observer** le journal causal complet (tout ce qui s'est passé et pourquoi). - **Intervenir** : révoquer des capacités, suspendre un agent, forcer un retour arrière. - **Autoriser** : signer les actions à fort impact mises en attente.

Il ne dispose pas d'un terminal interactif. Son interface est un client externe qui parle le même protocole que les agents, mais avec des droits privilégiés.

---

## 5. Les propriétés

---

Voici le cœur de la thèse. Le projet affirme qu'un système d'exploitation conçu pour le profil B peut **garantir par construction** des propriétés que Linux n'offre qu'au prix de couches applicatives coûteuses.

Une règle de méthode d'abord : **une propriété n'est retenue que si elle est vérifiable expérimentalement**. Une propriété qu'on ne sait pas mesurer est une intention, pas une propriété.

Il y a six propriétés, P1 à P6. Les trois « vitrines » sont P1, P2 et P3.

### P1 — Densité ×5

**Énoncé** : héberger au moins **cinq fois plus** d'agents *dormants* que Linux+conteneurs sur le même matériel.

💡 Un agent *dormant* attend un message sans rien faire. La question est : combien peut-on en empiler en mémoire avant de saturer la machine ?

**Pourquoi c'est plausible** : un conteneur Docker garde un interpréteur Python complet en mémoire même quand l'agent ne fait rien (~44 Mo/agent). La technologie WASM (voir section 6), elle, laisse les pages d'un agent dormant *virtuelles* (non chargées en mémoire physique) : ~9,65 Ko/agent. Le rapport mesuré est de **4 500 à 7 375×** — bien au-delà du ×5 visé.

**Statut** : **partiel**. Le rapport mémoire dépasse largement la cible, mais c'est mesuré sur Linux contre Docker, pas dans une comparaison stricte sur le matériel cible final. (Voir section 11 pour la nuance

honnête.)

## P2 — Retour arrière transactionnel $\leq 100$ ms

**Énoncé** : pouvoir revenir à n'importe quel état passé (depuis la dernière barrière de validation) en un temps borné,  $\leq 100$  ms pour 100 actions en arrière.

**Pourquoi c'est le différenciateur principal** : sans retour arrière propre, le système n'apporte rien de nouveau — il est seulement plus léger. Avec lui, un agent peut explorer, se tromper, et revenir en arrière proprement, ce qui est impossible nativement sous Linux.

⚠ **Une correction honnête du projet** : au départ, la spécification promettait une complexité  $O(\log N)$  (un coût qui croît *très* lentement avec le nombre d'actions). À l'épreuve, l'implémentation s'est révélée en  $O(\text{profondeur})$  — coût *linéaire* dans la profondeur du retour. La promesse  $O(\log N)$  a été **retirée** (et non maquillée), parce que la vraie garantie qui compte est la **borne en temps** ( $\leq 100$  ms), qui, elle, tient. Ce genre de rétractation assumée est un marqueur de méthode du projet.

💡 **Notation  $O(\dots)$  (complexité algorithmique)** : une façon de décrire comment le coût d'une opération grandit avec la taille du problème.  $O(N)$  = coût proportionnel à  $N$  (deux fois plus de données → deux fois plus lent).  $O(\log N)$  = coût qui croît très lentement (mille fois plus de données → environ dix fois plus lent seulement). «  $O(\text{profondeur})$  » signifie que le coût dépend du nombre de pas qu'on remonte, pas du total des données.

**Statut : conforme (PASS)**. Mesuré : 17–20 ms pour revenir 500 actions en arrière — cinq fois sous la cible.

## P3 — Traçabilité causale $\leq 10$ ms

**Énoncé** : retrouver n'importe quelle action par son identifiant en **p99  $\leq 10$  ms**, même sur un journal de  $10^8$  (cent millions) d'actions.

💡 **p99 (99<sup>e</sup> centile)** : si 99 % des recherches sont plus rapides que 10 ms, on dit « p99  $\leq 10$  ms ». C'est plus exigeant que « 10 ms en moyenne » : cela borne aussi les cas lents. On mesure le p99 parce qu'une garantie ne vaut que par son pire cas courant, pas par sa moyenne. (Un *centile* est une coupure : le 99<sup>e</sup> centile est la valeur en dessous de laquelle tombent 99 % des mesures.)

**Pourquoi c'est plausible** : retrouver une donnée par sa clé dans un index bien construit est l'opération la plus optimisée des bases de données. Le projet utilise RocksDB (voir section 6), taillé exactement pour cela.

**Statut : conforme (PASS)** sur Linux en lecture seule : p99 de **1,4 à 1,9 ms** sur  $10^8$  actions — cinq à sept fois sous la cible.

⚠ Le projet est précis sur la **portée** de cette garantie. La borne 10 ms vaut pour une *recherche isolée sur une base statique* (appelée **P3a**). Le cycle complet « écrire puis relire avec garantie de durabilité » (**P3b**) a une borne distincte ( $\leq 20$  ms). Et le régime multi-agent sous forte concurrence (**P3c**) a des bornes encore plus larges. Confondre ces portées, c'est revendiquer plus qu'on n'a mesuré.

## P4 — Isolation par capacités

**Énoncé** : tout accès à une ressource exige une capacité explicite. Aucun accès par défaut. Et la délégation respecte l'**atténuation** (un enfant n'a jamais plus de droits que son parent), sur deux axes : les *permissions* (lecture seule vs lecture+écriture) et la *portée* (tout le magasin vs un seul sous-dossier).

Trois conditions doivent tenir *ensemble* : 1. **Les accès autorisés passent** (100 %). 2. **Les accès non autorisés échouent** (100 %, sans contournement possible). 3. **Les refus sont audités** (consignés dans le journal causal).

**Statut : conforme (PASS)** — y compris contre une attaque réelle (voir « adjoint confus », section 10).

## P5 — Déterminisme de transition

**Énoncé** : deux copies du même agent, parties du même état et nourries de la même séquence de messages, produisent la **même** suite de messages et le **même** état final.

💡 Pourquoi est-ce précieux ? Parce que cela rend les bugs **rejouables**. Si un agent a planté hier, on peut rejouer exactement sa séquence aujourd'hui et observer le plantage. Sur un système non déterministe, un bug sur deux est irreproductible.

💡 **Déterministe vs stochastique** : un processus *déterministe* donne toujours le même résultat à partir des mêmes entrées. Un processus *stochastique* comporte une part d'aléa (les grands modèles de langage en sont : la même question peut produire deux réponses différentes).

Pour que cela fonctionne, toutes les sources de hasard (l'horloge, l'aléatoire, le résultat stochastique de l'IA, les données externes) doivent passer par des **primitives substituables** — des points d'entrée que le système peut remplacer en mode rejou.

⚠ C'est une garantie **conditionnelle** : elle tient *si* le substrat empêche la mémoire partagée non médiée (sinon, du hasard invisible s'y glisserait). C'est pourquoi la concurrence à l'intérieur d'un agent est interdite.

**Statut : conforme conditionnel (PASS conditionnel).**

## P6 — Atomicité face aux pannes

**Énoncé** : si l'agent est interrompu brutalement *au milieu* d'une transaction, après redémarrage l'état est **soit** celui d'avant la transaction, **soit** celui d'après la dernière barrière de validation — **jamais un entre-deux**.

 **Atomicité** : la propriété du « tout ou rien ». Une opération atomique se produit entièrement ou pas du tout, jamais à moitié. Ici, appliquée à l'état complet d'un agent : pas d'état « à moitié écrit » après une panne.

**Statut** : conforme au niveau processus (PASS niveau processus). Vérifié par 40 scénarios où le programme est interrompu à 4 instants différents : dans 100 % des cas, l'état après redémarrage est cohérent.

 **Limite assumée** : « interrompu brutalement » couvre le signal SIGKILL (l'interruption immédiate et non négociable d'un programme par le système), un plantage du programme, ou l'intervention du tueur de mémoire (en anglais OOM-killer, qui tue un programme quand la mémoire vive est épuisée). Cela ne couvre **pas** la coupure de courant ni le plantage du noyau — dans ces cas, le cache du système d'exploitation est perdu. Cette extension demande un test sur matériel réel (impossible en émulation) et est documentée comme un trou connu, non dissimulé.

### L'ordre de priorité : que sacrifie-t-on en premier ?

Les propriétés peuvent entrer en tension (par exemple, plus on trace finement, plus on ralentit, donc P3 contre P1). Il faut donc un ordre d'arbitrage. C'est la décision **ADR-0001** :

**P4 > P2 > P3 > P6 > P5 > P1**

Cela se lit : « s'il faut céder sur une propriété, on cède d'abord sur P1 (la densité), en dernier sur P4 (l'isolation) ».

**Justification de l'ordre** : - **P4 (isolation) en premier** : sans contrôle d'accès, héberger des agents stochastiques est carrément dangereux. C'est non négociable. - **P2 (retour arrière) ensuite** : c'est le différenciateur fonctionnel. Sans lui, le système n'apporte rien. - **P3 (traçabilité)** : on peut assouplir sa vitesse (10 ms → 100 ms) sans tuer le projet, mais pas sa *correction*. - **P6 (atomicité)** : largement un corollaire de P2, rarement en tension. - **P5 (déterminisme)** : précieux mais dégradable en mode « au mieux ». - **P1 (densité) en dernier** : c'est une cible chiffrée. Atteindre ×4 au lieu de ×5 n'invalide pas la thèse, tant que le reste tient.

 **La correction prime sur la performance**. C'est la philosophie résumée par cet ordre. Un système rapide mais incorrect ne vaut rien ; un système correct mais un peu moins dense reste utile.

## Les deux « régimes »

Une subtilité importante pour ne pas se tromper en lisant les résultats :

- **Régime R1 (« Effets »)** : P2, P3, P4, P6. Actif **partout**, y compris quand l'IA fonctionne sur un service externe (dans le nuage, en anglais *cloud*).
- **Régime R2 (« Ressources »)** : P1, P5. Actif **seulement** quand l'inférence IA fonctionne *en local* (l'environnement d'exécution contrôle le modèle).

⚠ **Ne jamais revendiquer les six propriétés sans nommer le régime.** P1 et P5 n'ont de sens que si l'environnement d'exécution maîtrise le fonctionnement de l'IA. Si l'IA est appelée à distance, l'environnement ne contrôle ni la densité ni le déterminisme de cette partie.

## 6. Architecture

Voici l'agencement des pièces, de haut (l'agent) en bas (le matériel) :

```
Agent (un module .wasm)
| s'exprime via une ABI = des "fonctions hôtes"
| (agent_infer, emit, agent_add_cause, ...)
▼
Environnement d'exécution en Rust / Tokio ← le cœur du projet (poc/runtime/)
| Ordonnanceur ..... répartit la capacité d'inférence (réservoir borné)
| CausalLog ..... le journal des actions (RocksDB)
| ContentStore ..... les états sauvegardés (RocksDB, DAG de Merkle)
| Capacités ..... l'arbre des droits (délégation, révocation)
▼
Substrat : Linux (actuel) · seL4 (cible)
```

Décortiquons chaque brique et chaque technologie.

### WebAssembly (WASM) et Wasmtime — l'isolation des agents

**WebAssembly (WASM)** est un format de programme compilé, portable et confiné dans un **bac à sable** (en anglais *sandbox*) : un programme WASM s'exécute dans une « bulle » qui ne peut toucher que ce qu'on l'autorise explicitement à toucher. À l'origine, ce format a été prévu pour faire fonctionner du code dans les navigateurs en sécurité.

**Wasmtime** est le moteur qui exécute ces programmes WASM hors d'un navigateur.

**Pourquoi WASM pour les agents ?** Deux raisons : 1. **Légèreté** (P1) : un agent WASM dormant coûte ~9,65 Ko, contre ~44 Mo pour un conteneur Docker+Python. C'est le fondement de la densité ×5. 2.

**Isolation** (P4) : la bulle WASM ne peut appeler que les fonctions que l'environnement lui expose —

support naturel du modèle « non-ambient » des capacités.

💡 **ABI et fonctions hôtes** : l'agent (dans sa bulle WASM) ne peut pas appeler Linux. Il ne peut appeler que les fonctions que l'environnement lui tend par un orifice précis dans la bulle : ce sont les **fonctions hôtes** (en anglais *host functions*). La liste de ces fonctions et leur format constitue l'**ABI** (*Application Binary Interface*, l'interface binaire de programmation). Exemples : `agent_infer` (« réalise une inférence IA »), `emit` (« publie un message / pose une barrière »), `agent_self_rollback` (« reviens en arrière »).

## Tokio — le moteur asynchrone

**Tokio** est une bibliothèque du langage Rust pour écrire du code **asynchrone** : gérer des milliers de tâches en attente (d'un message, d'une réponse) sans bloquer, avec très peu de fils d'exécution système. C'est ce qui permet d'héberger beaucoup d'agents dormants à faible coût.

## Rust — le langage

Tout l'environnement d'exécution est écrit en **Rust**, un langage qui garantit dès la compilation l'absence de toute une classe de bugs mémoire (accès hors limites, usage après libération) **sans ramasse-miettes**. Pour un composant de type système d'exploitation devant être à la fois sûr et rapide, c'est le choix de référence aujourd'hui.

## RocksDB — la base de stockage

**RocksDB** est une base de données clé-valeur très rapide, embarquée (sans serveur séparé). Le projet l'utilise pour **deux** bases distinctes : le CausalLog et le ContentStore.

Pourquoi RocksDB plutôt que SQLite ? Parce que RocksDB repose sur un **arbre LSM**, le bon outil pour ce besoin :

💡 **Arbre LSM (*Log-Structured Merge tree*)** : une structure de données optimisée pour *écrire beaucoup et vite* (on ajoute toujours à la fin, jamais au milieu), puis *fusionner* en arrière-plan — opération appelée **compaction**. À l'opposé, un **arbre B** (en anglais *B-tree*, utilisé par SQLite) est optimisé pour des lectures/écritures dispersées. Pour un journal qui ne fait qu'ajouter  $10^8$  entrées et les relire par clé, l'arbre LSM est architecturalement le bon choix.

💡 **Filtre de Bloom (*Bloom filter*)** : un petit filtre probabiliste qui répond très vite à la question « cette clé est-elle *certainement absente* ? ». S'il répond « absente », inutile d'aller lire le disque. C'est ce qui rend quasi gratuites les recherches de clés inexistantes, et qui aide à tenir le p99 de P3.

## CausalLog — le journal des actions

Le **CausalLog** (« journal causal », poc/causal-log/) est le journal en **ajout seul** (en anglais *append-only* : on ne fait qu'ajouter, jamais modifier) de toutes les actions.

- **Clé** = l'action\_id = l'empreinte SHA-256 de l'entrée (donc adressée par le contenu).
- **Valeur** = la LogEntry : { agent\_id, numéro de séquence, horodatage, parent\_ids[], empreinte de l'état après, charge utile }.

Le champ crucial est **parent\_ids[]** : la **liste** des actions ayant directement causé celle-ci. C'est ce qui forme le **DAG causal** (voir Pari 1, section 8).

## ContentStore — les états sauvegardés

Le **ContentStore** (« magasin de contenu », poc/store/) conserve les **instantanés** (snapshots) de l'état des agents, sous forme de **DAG de Merkle**.

💡 **DAG de Merkle (Merkle DAG)** : un graphe où chaque nœud est adressé par l'empreinte de son contenu, et où chaque nœud référence ses parents par leur empreinte. Conséquence : pour revenir en arrière, il suffit de remonter la chaîne parent → parent → ... jusqu'à la cible. C'est exactement la structure de Git. C'est ce qui rend le retour arrière (P2) réalisable sans recopier tout l'état à chaque fois.

Chaque instantané est un SnapshotHeader : { data\_hash, parent (facultatif), seq, ts }. Le retour arrière parcourt cette chaîne, une lecture RocksDB par maillon (d'où le O(profondeur)).

💡 **La discipline « no-force »** (ADR-0027) : le ContentStore peut être *en avance* sur le journal (un état sauvegardé que le journal ne référence pas encore — c'est un « orphelin », inoffensif, du déchet à ramasser plus tard). Mais il ne doit **jamais être en retard** (le journal référençant un état absent du magasin — cela, c'est une corruption, dite « référence pendante »). Cette asymétrie est ce qui rend P6 tenable sans payer une synchronisation disque coûteuse à chaque action.

💡 **Synchronisation disque forcée (fsync)** : une commande qui oblige le système à écrire *réellement* sur le disque ce qui n'était encore que dans un cache en mémoire. C'est lent mais c'est la seule garantie qu'une donnée survivra à une coupure de courant. La discipline « no-force » consiste justement à *éviter* ce coût sur le chemin courant, en s'appuyant sur le cache du système d'exploitation, qui survit à une simple interruption de programme.

## Capacités — l'arbre des droits

Le module **Capacités** (poc/capabilities/) maintient l'arbre de délégation : qui a donné quel droit (réduit) à qui. Révoquer un nœud invalide récursivement tout son sous-arbre — d'où le coût

O(profondeur) de la révocation.

## Ordonnanceur et réservoir d'inférence — partager l'IA

L'**inférence** (faire fonctionner le modèle de langage pour produire une réponse) est une **ressource rare et coûteuse** : sur processeur central (CPU), un cycle prend de 6 à 18 secondes. Plusieurs agents se la partagent. L'**ordonnanceur** (en anglais *scheduler*) gère un **réservoir d'inférence** (en anglais *inference pool*) : un sémaphore de capacité *k* (seulement *k* inférences à la fois).

💡 **Sémaphore** : un compteur de jetons. Pour inférer, un agent doit prendre un jeton ; il le rend quand il a terminé. S'il n'y a plus de jeton, il attend. C'est ce qui *borne* le nombre d'inférences simultanées.

Deux garanties importantes : - **Priorité** : *Foreground* (« premier plan ») passe avant *Batch* (« arrière-plan »). - **Anti-famine (équité)** : un travail *Batch* qui attend trop longtemps est promu *Foreground*, pour ne jamais être oublié indéfiniment. (Sans cela, un flot de travaux prioritaires affamerait éternellement les travaux de fond.)

---

## 7. Parcours d'une action

Assemblons les pièces. Voici, simplifié, ce qui se produit lorsqu'un agent traite un message (le « cycle W1 » utilisé dans les mesures étalons) :

1. **Réception.** Un message arrive dans la boîte de réception (la file Tokio) de l'agent. ← *c'est une action (réception).*
2. **Introspection.** L'agent appelle `agent_introspect()` pour lire son état courant (numéro de séquence, cycle de vie).
3. **Inférence.** L'agent appelle `agent_infer(prompt)`. L'ordonnanceur lui attribue un jeton du réservoir d'inférence (ou le fait patienter). Le modèle réfléchit.
4. **Barrière de validation.** L'agent appelle `commit()` : il pose un point de non-retour. À partir d'ici, tout ce qui précède devient non annulable.
5. **Émission.** L'agent appelle `emit(message)` pour publier un résultat. ← *c'est une action (émission).*

À chaque action, **en coulisses** : - une `LogEntry` est ajoutée au **CausalLog** (avec son `action_id` = empreinte, ses `parent_ids`, l'empreinte de l'état après) → cela alimente P3 (traçabilité) ; - un instantané peut être écrit dans le **ContentStore** → cela alimente P2 (retour arrière) ; - chaque accès à une ressource est vérifié contre une **capacité** → cela alimente P4 (isolation) ; - toutes les sources de



hasard passent par des **primitives substituables** → cela alimente P5 (déterminisme).

Et si le processus est **interrompu brutalement** entre l'étape 4 et l'étape 5 ? Au redémarrage, RocksDB rejoue son journal d'écriture anticipée (en anglais *Write-Ahead Log*, abrégé WAL — le carnet où chaque modification est notée *avant* d'être appliquée), et l'état revient soit à « avant la transaction », soit à « après la dernière barrière de validation » — jamais au milieu. → c'est P6 (atomicité).

 **Le point à retenir** : les six propriétés ne sont pas six modules séparés. Elles émergent de **la même infrastructure partagée** (journal adressé par le contenu + magasin de Merkle + arbre de capacités + primitives substituables). C'est pourquoi ajouter P4, P5 et P6 à P1, P2 et P3 ne triple pas le coût : tout est mutualisé. (C'est l'objet des « synergies » de la spécification.)

---

## 8. Les cinq paris

---

Le projet repose sur cinq paris d'architecture. Chacun est **falsifiable** : la condition qui le réfuterait a été écrite *avant* l'expérience.

### Pari 1 — Un DAG causal, pas un arbre

 **DAG** = *Directed Acyclic Graph*, « graphe orienté sans cycle ». Un **arbre** est le cas particulier où chaque nœud n'a *qu'un seul* parent. Un **DAG** est plus général : un nœud peut avoir *plusieurs* parents. (« Sans cycle » signifie qu'on ne peut pas revenir à son point de départ en suivant les flèches.)

**Le pari** : la causalité réelle entre actions d'agents parallèles est un DAG, pas un arbre. Lorsque deux sous-agents travaillent en parallèle et que leurs résultats fusionnent, l'action de fusion a **deux** parents. Un arbre (un seul parent) forcerait à sérialiser artificiellement, ce qui mentirait sur la véritable causalité.

C'est pourquoi chaque action porte une liste de parents (`caused_by[]` / `parent_ids[]`). **Validé** dès les premières expériences multi-agents.

### Pari 2 — RocksDB (LSM), pas SQLite (arbre B)

**Le pari** : pour un journal en ajout seul de  $10^8$  entrées avec recherche par clé opaque, l'arbre LSM est le bon outil (voir section 6). **Validé** :  $p99 \leq 2$  ms sur  $10^8$  entrées, cinq à sept fois sous la cible.

### Pari 3 — Wasmtime + Tokio, pas Docker

**Le pari** : l'isolation par module WASM est des milliers de fois plus légère par agent dormant que Docker+Python. **Validé** : 4 500 à 7 375 fois moins de mémoire vive.

Et le coût a même été *modélisé* :  $\text{surcoût}(N) = 9,65 - 54/N$  Ko par agent (avec une qualité d'ajustement  $R^2=0,988$ ). Traduction : le coût asymptotique est  $\sim 9,65$  Ko/agent ; le terme  $54/N$  représente des coûts fixes partagés (le binaire WASM, le moteur Tokio) qui s'amortissent dès  $N \geq 300$  agents.

  **$R^2$  (coefficient de détermination)** : un indicateur entre 0 et 1 disant à quel point une formule colle aux données mesurées. 0,988 = la formule explique 98,8 % de la variation observée : excellent. (« Asymptotique » désigne la valeur vers laquelle on tend quand N devient très grand.)

## Pari 4 — Capacités révocables, pas identité Unix

**Le pari** : déléguer des jetons de droits précis et révocables vaut mieux que le modèle « root a tout ».

**Validé** fonctionnellement, y compris contre l'attaque de l'« adjoint confus » (section 10).

## Pari 5 — Supervision asymétrique, pas temps réel

**Le pari** : le superviseur humain observe, intervient, autorise — mais **pas en temps réel**. Cela dimensionne tout le reste : la cible de latence de P3 est 10 ms (« un humain peut attendre »), pas 1  $\mu$ s (« une interruption matérielle ne peut pas attendre »). Corollaire : le journal est conçu **durabilité d'abord**, pas latence d'abord.

---

## 9. Les substrats

Le projet a été construit sur **deux** substrats successifs. Comprendre pourquoi est essentiel.

 **Substrat** : la couche d'exécution située *sous* l'environnement d'exécution — le noyau réel sur lequel tout repose.

### Linux (substrat actuel)

Toute la validation fonctionnelle (phases 5 à 7) a été réalisée sur Linux. Les primitives y sont réelles et mesurées. **Mais** l'isolation y est purement *logicielle* (le bac à sable Wasmtime). Si quelqu'un trouve une faille dans Wasmtime, il sort de la bulle — et sous Linux, tous les agents partagent le même processus système, donc une évocation compromet tout.

 Les mesures faites sur Linux sont valides **sur Linux**, mais **non transférables** au substrat cible. C'est une décision assumée, pas un oubli : mesurer la densité sur Linux ne dit rien de la densité sur seL4.

## seL4 (substrat cible)

seL4 est un **micronoyau formellement vérifié**. Deux notions à décoder :

💡 **Micronoyau (*microkernel*)** : un noyau minimaliste qui ne fait *que* le strict nécessaire (gestion mémoire, communication entre processus), en laissant le reste à des programmes en espace utilisateur. Plus petit = plus facile à vérifier.

💡 **Formellement vérifié** : on a **prouvé mathématiquement** (preuve elle-même contrôlée par machine, pas simplement « testée ») que le code du noyau est conforme à sa spécification. seL4 est l'un des très rares noyaux au monde dans ce cas. Cela en fait une base de confiance d'un tout autre niveau que Linux (~30 millions de lignes non prouvées).

💡 **Base de calcul de confiance (*Trusted Computing Base, TCB*)** : l'ensemble du code en qui l'on *doit* avoir confiance pour que la sécurité tienne. Plus elle est petite et prouvée, mieux c'est. Tout l'argument seL4 du projet consiste à réduire et prouver cette base.

**Ce qui a été porté sur seL4** (11 jalons, C.1 à C.11, sur l'émulateur QEMU pour processeur ARM 64 bits, dit AArch64) :

💡 **QEMU** : un émulateur — un logiciel qui simule un autre ordinateur (ici, une machine ARM) sur la machine de développement. Pratique pour tester sans matériel physique, mais inadapté pour mesurer des vitesses réelles d'accès disque.

💡 **virtio-blk** : un disque *virtuel* normalisé fourni par l'émulateur au système invité. Le store seL4 écrit dessus comme sur un vrai disque.

- Faire fonctionner Wasmtime *sans bibliothèque standard* (`no_std` en Rust — un défi technique réel, car la plupart du code suppose un système d'exploitation complet sous lui).
- Un magasin persistant (`redb`, une base clé-valeur, écrivant sur le disque virtuel `virtio-blk`).
- L'application du principe **W^X** sur le compilateur à la volée (voir ci-dessous).
- La validation multi-agents avec un « badge » de capacité par agent.
- L'atomicité face aux pannes sous des scénarios d'interruption adverses.
- L'isolation prouvée contre des modules WASM **malveillants** (qui tentent un accès hors limites, une division par zéro, une boucle infinie).

💡 **W^X (*Write XOR eXecute*, « écriture OU BIEN exécution »)** : une zone mémoire est soit *modifiable*, soit *exécutable*, jamais les deux à la fois. Cela empêche un attaquant d'écrire du code puis de l'exécuter. C'est délicat pour un compilateur **à la volée** (en anglais *Just-In-Time*, abrégé JIT — qui génère du code machine pendant l'exécution), parce que celui-ci doit justement écrire *puis* exécuter — d'où la bascule contrôlée des permissions au moment voulu.

Le prototype seL4 a été **clos** (ADR-0049) une fois ces jalons atteints.

---

## 10. La méthode

---

Ce qui distingue ce projet d'un simple prototype, c'est sa **discipline de méthode**. Cinq engagements.

### Les décisions sont explicites et versionnées (les ADR)

💡 **ADR** = *Architecture Decision Record*, « fiche de décision d'architecture ». Chaque décision importante est consignée dans un fichier numéroté (`decisions/0001-...`, `0002-...`) précisant : *quoi* a été décidé, *pourquoi*, *quelles alternatives* ont été écartées, et *sous quelles conditions* il faudrait y revenir.

Il y en a 56 (dans `decisions/`). Un ADR est **contraignant** : tant qu'il n'a pas été amendé ou remplacé, on doit s'y conformer. Un prototype qui s'en écarte est une **dette à tracer**, pas un précédent à suivre.

💡 Pourquoi est-ce puissant ? Lorsqu'un résultat d'expérience a contredit une hypothèse (par exemple une fausse alerte de fuite mémoire), le système d'ADR a fourni le cadre pour décider rationnellement — réfuter l'hypothèse, amender le design, ou ajuster la mesure. Rien n'est enterré en douce.

### Les hypothèses sont écrites *avant* l'expérience

Chaque mesure étalon a une **cible définie à l'avance**. Un résultat qui bat la cible de 17 fois n'a de valeur que *parce que* la cible était fixée avant. Choisir la cible après coup ne prouverait rien.

### Les échecs sont documentés, pas enterrés

Trois « **ADR de réfutation** » (0032, 0033, 0034) documentent des cas où les données ont contredit le modèle initial. Exemple : un test d'endurance (« T6-soak ») semblait montrer une fuite mémoire (la mémoire vive grimpait). Enquête : le critère statistique employé (une régression linéaire) était structurellement inapplicable à cause des pics de compaction RocksDB ( $R^2=0,24$ , donc l'ajustement ne signifiait rien). L'hypothèse de fuite a été **explicitement réfutée** : la mémoire est en réalité bornée. La bonne réponse était de corriger le modèle, pas de jeter la donnée.

### On refuse les mesures « sur le mauvais substrat »

Une mesure qui ne prouve rien sur la cible n'est pas une mesure utile. Mesurer P3a sur le stockage émulé de QEMU ne dirait rien de la latence réelle sur du vrai matériel seL4. Le projet a donc **refusé**

cette mesure (elle attend du matériel physique), plutôt que de publier un chiffre trompeur. Cela a coûté une mesure, et gagné en intégrité.

## On attaque le système, on ne se contente pas de le valider

💡 « Une propriété qui ne tient que sous des entrées amicales n'est pas une propriété. »

Le système a été soumis à des **campagnes adverses** (ADR-0050/0051) : on attaque, on ne se borne pas à vérifier. L'exemple emblématique est celui de l'« **adjoint confus** » :

💡 **Adjoint confus (*confused deputy*)** : une attaque classique où un acteur malveillant, dépourvu des droits, manipule un intermédiaire *légitime* (qui, lui, détient les droits) pour réaliser à sa place l'action interdite. L'adjoint est « confus » : il agit avec ses propres droits, sans réaliser qu'il sert un attaquant.

Résultat : l'isolation a **tenu** (l'attaque n'a pas réussi à élever ses privilèges). **Mais** la campagne a révélé un *trou d'audit* : en inondant le système de plus de 100 refus bénins par seconde, un attaquant pouvait *masquer* un refus malveillant dans le bruit. Ce n'était pas un défaut d'isolation (le droit restait correctement refusé), mais un défaut d'**observabilité**. Le trou a été **corrigé** (agrégation des refus par ressource) et le correctif **re-testé** avec une seconde attaque. C'est précisément le cycle visé : attaquer → trouver → corriger → re-tester.

---

## 11. Bilan

L'honnêteté sur ce qui est acquis et ce qui ne l'est pas est une valeur centrale du projet.

### Ce qui est démontré

| Propriété                          | Statut                            | Preuve                                        |
|------------------------------------|-----------------------------------|-----------------------------------------------|
| <b>P2</b> Retour arrière           | ✅ conforme                        | 17–20 ms pour profondeur=500 (cible ≤ 100 ms) |
| <b>P3a</b> Traçabilité (recherche) | ✅ conforme (Linux, lecture seule) | p99 1,4–1,9 ms sur 10 <sup>8</sup> actions    |
| <b>P4</b> Isolation                | ✅ conforme                        | tient sous l'attaque de l'adjoint confus      |
| <b>P5</b> Déterminisme             | ✅ conforme (conditionnel)         | rejeu de 1 000 messages identique             |
| <b>P6</b> Atomicité face aux       | ✅ conforme (niveau                | 40 scénarios d'interruption, 100 %            |

| Propriété        | Statut     | Preuve                               |
|------------------|------------|--------------------------------------|
| pannes           | processus) | cohérents                            |
| Intégration seL4 | ✅ C.1–C.11 | 11 jalons sur émulateur QEMU AArch64 |

### Ce qui n'est PAS démontré (et pourquoi, de façon assumée)

- **P1 quantifié face à Linux+conteneurs (au sens strict)** : le rapport mémoire dépasse largement la cible ( $\times 4\ 500$  et plus), mais la comparaison stricte « à latence d'action équivalente sur le substrat cible » n'a pas été établie — parce que les chiffres Linux ne sont **pas transférables** à seL4. *Décision explicite, pas mesure manquante.*
- **P3a sur vrai matériel seL4 (« D-P3a »)** : le banc de mesure est prêt, mais il faut une **carte ARM physique** (ou un accès disque direct, *passthrough* NVMe) — l'accès disque émulé de QEMU n'est pas un substrat de mesure recevable. *Bloqué par le matériel.*
- **Durabilité face à la coupure de courant** : hors périmètre. L'oracle de cohérence est écrit, en attente du même déclencheur matériel.
- **Atomicité inter-magasins** : il subsiste une fenêtre étroite où, sous perte de cache, le journal et le magasin (deux bases RocksDB séparées) pourraient diverger. Atténuée par une sécurité de repli au moment de la restauration (qui détecte le problème), pas fermée. La fermeture (validation atomique entre les deux) est rattachée au futur chantier de ramasse-miettes.

💡 La leçon de méthode à retenir de ce bilan : « **non démontré** » n'est pas « **échoué** ». Le projet distingue rigoureusement *mesuré*, *planifié*, *différé pour raison matérielle*, et *abandonné parce que non transférable*. Confondre ces quatre statuts reviendrait à mentir sur l'état réel.

## 12. Glossaire de poche

| Terme                               | Définition en une ligne                                                   |
|-------------------------------------|---------------------------------------------------------------------------|
| Système d'exploitation              | Programme qui arbitre l'accès des programmes au matériel                  |
| Noyau (kernel)                      | Le cœur tout-puissant du système d'exploitation                           |
| Environnement d'exécution (runtime) | Programme qui héberge et sert d'autres programmes (ici : les agents)      |
| Agent                               | Programme piloté par IA, autonome, identifié par un <code>agent_id</code> |

| Terme                                             | Définition en une ligne                                                 |
|---------------------------------------------------|-------------------------------------------------------------------------|
|                                                   | permanent                                                               |
| <b>Acteur</b>                                     | Entité qui reçoit/traite/émet des messages, un à la fois                |
| <b>Action</b>                                     | Un message reçu ou émis — l'unité de base du projet                     |
| <b>Capacité (capability)</b>                      | Jeton de droit précis : non-ambient, délégable, atténuable, révocable   |
| <b>Atténuation</b>                                | Une capacité dérivée a <i>au plus</i> les droits de son parent          |
| <b>Barrière de validation (commit barrier)</b>    | Point de non-retour ; après elle, plus de retour arrière                |
| <b>Transaction</b>                                | Suite d'actions entre deux barrières de validation (tout ou rien)       |
| <b>Retour arrière (rollback)</b>                  | Restauration de l'état local à un instant passé                         |
| <b>État local</b>                                 | Acteurs locaux + magasin local + messages internes (exclut l'externe)   |
| <b>Adressé par le contenu (content-addressed)</b> | Donnée rangée à une adresse = l'empreinte de son contenu                |
| <b>Empreinte / hachage (hash, SHA-256)</b>        | Empreinte de taille fixe d'une donnée (32 octets ici)                   |
| <b>DAG</b>                                        | Graphe orienté sans cycle ; un nœud peut avoir plusieurs parents        |
| <b>DAG de Merkle</b>                              | DAG où chaque nœud est adressé par l'empreinte de son contenu (cf. Git) |
| <b>Instantané (snapshot)</b>                      | Photographie de l'état d'un agent à un instant donné                    |
| <b>CausalLog</b>                                  | Le journal en ajout seul des actions (RocksDB)                          |
| <b>ContentStore</b>                               | Le magasin des instantanés d'état (RocksDB, DAG de Merkle)              |
| <b>WASM / WebAssembly</b>                         | Format de programme portable, confiné dans un bac à sable               |

| Terme                                     | Définition en une ligne                                                           |
|-------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Bac à sable (sandbox)</b>              | Environnement isolé où un programme ne touche que l'autorisé                      |
| <b>Wasmtime</b>                           | Moteur qui exécute du WASM hors navigateur                                        |
| <b>ABI / fonction hôte</b>                | Les fonctions que l'environnement expose à l'agent dans sa bulle                  |
| <b>Tokio</b>                              | Bibliothèque Rust pour le code asynchrone (beaucoup de tâches en attente)         |
| <b>RocksDB</b>                            | Base clé-valeur embarquée, rapide (arbre LSM)                                     |
| <b>Arbre LSM</b>                          | Structure optimisée pour écrire beaucoup et fusionner en arrière-plan             |
| <b>Arbre B (B-tree)</b>                   | Structure optimisée pour accès dispersés (SQLite) — écartée ici                   |
| <b>Compaction</b>                         | Fusion en arrière-plan des écritures d'un arbre LSM                               |
| <b>Filtre de Bloom</b>                    | Filtre qui dit vite si une clé est certainement absente                           |
| <b>Ordonnanceur (scheduler)</b>           | Composant qui répartit la capacité d'inférence entre agents                       |
| <b>Réservoir d'inférence (pool)</b>       | Sémaphore bornant le nombre d'inférences IA simultanées                           |
| <b>Sémaphore</b>                          | Compteur de jetons d'accès à une ressource limitée                                |
| <b>Inférence</b>                          | Faire fonctionner le modèle de langage pour produire une réponse (ressource rare) |
| <b>Synchronisation disque (fsync)</b>     | Forcer l'écriture réelle sur disque (lent, mais sûr face à la coupure)            |
| <b>Journal d'écriture anticipée (WAL)</b> | Carnet où chaque modification est notée avant d'être appliquée                    |
| <b>p99 / centile</b>                      | 99 <sup>e</sup> centile : 99 % des cas sont plus rapides que cette valeur         |



| Terme                                    | Définition en une ligne                                                                        |
|------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>Complexité O(...)</b>                 | Façon de décrire comment un coût grandit avec la taille du problème                            |
| <b>Déterministe / stochastique</b>       | Toujours le même résultat / comportant une part d'aléa                                         |
| <b>Atomicité</b>                         | Propriété du « tout ou rien » : entièrement, ou pas du tout                                    |
| <b>Superviseur asymétrique</b>           | Le rôle humain : observe, intervient, autorise — pas en temps réel                             |
| <b>Profil B</b>                          | L'agent cible : 1 h – 1 mois, état persistant, $10^4$ – $10^8$ actions, supervision ponctuelle |
| <b>Substrat</b>                          | La couche d'exécution sous l'environnement (Linux ou seL4)                                     |
| <b>seL4</b>                              | Micronoyau formellement vérifié (substrat cible)                                               |
| <b>Micronoyau (microkernel)</b>          | Noyau minimaliste, plus facile à prouver                                                       |
| <b>Formellement vérifié</b>              | Prouvé mathématiquement conforme à sa spécification (pas seulement testé)                      |
| <b>Base de calcul de confiance (TCB)</b> | Le code en qui l'on doit avoir confiance                                                       |
| <b>W^X</b>                               | Mémoire soit modifiable, soit exécutable, jamais les deux                                      |
| <b>Compilation à la volée (JIT)</b>      | Génération de code machine pendant l'exécution                                                 |
| <b>no_std</b>                            | Code Rust sans bibliothèque standard (requis sur seL4)                                         |
| <b>QEMU</b>                              | Émulateur qui simule une autre machine (ici ARM)                                               |
| <b>AArch64</b>                           | Architecture de processeur ARM 64 bits (la cible seL4)                                         |
| <b>virtio-blk</b>                        | Disque virtuel normalisé fourni par l'émulateur                                                |
| <b>ADR</b>                               | Fiche de décision d'architecture (quoi, pourquoi, alternatives, conditions)                    |
| <b>Adjoint confus (confused deputy)</b>  | Attaque manipulant un intermédiaire légitime pour ses droits                                   |
| <b>no-force</b>                          | Discipline : le magasin peut être en avance sur le journal,                                    |

| Terme          | Définition en une ligne                                            |
|----------------|--------------------------------------------------------------------|
|                | jamais en retard                                                   |
| Régime R1 / R2 | R1 (effets, partout) / R2 (ressources, inférence locale seulement) |

---

## Pour aller plus loin

---

| Pour...                               | Consulter...                             |
|---------------------------------------|------------------------------------------|
| La vision et le problème en détail    | <code>spec/01-vision.md</code>           |
| La définition formelle des propriétés | <code>spec/02-properties.md</code>       |
| Le glossaire complet et nuancé        | <code>spec/06-glossary.md</code>         |
| Le résumé exécutif (en anglais)       | <code>OVERVIEW.md</code>                 |
| La vue technique condensée            | <code>wiki/01-README-technique.md</code> |
| Toutes les décisions d'architecture   | <code>decisions/INDEX.md</code> (56 ADR) |
| Les leçons empiriques                 | <code>lab/LESSONS.md</code> (L1–L119)    |
| L'état d'avancement et les dettes     | <code>TODO.md</code>                     |
| La version anglaise de ce guide       | <code>docs/learning-guide.en.md</code>   |
| ...                                   |                                          |